

Graph Retrieval-Augmented Generation untuk Meningkatkan Presisi *Retrieval* dan Validitas Sintaksis pada Konfigurasi Kubernetes

JIHAN AURELIA¹ and DIMITRI MAHAYANA¹

¹School of Electrical Engineering and Informatics, Institut Teknologi Bandung, Bandung 40132, Indonesia

Penulis koresponden: Jihan Aurelia (e-mail: 18222001@std.stei.itb.ac.id).

Naskah ini merupakan ringkasan Tugas Akhir Program Studi Sistem dan Teknologi Informasi, Institut Teknologi Bandung.

I. PENDAHULUAN

KUBERNETES [1] telah mengukuhkan posisinya sebagai standar industri *de facto* untuk orkestrasi kontainer, didorong oleh ekosistem yang matang dan fleksibilitas pengelolaan infrastruktur secara deklaratif [2], [3]. Namun demikian, paradigma deklaratif ini menuntut pengembang menulis konfigurasi YAML yang verbos dan hierarkis, dengan interdependensi antarobjek yang kuat namun sering kali tidak tersurat secara eksplisit [4]. Akibat kompleksitas ini, sekitar 80% insiden operasional Kubernetes berakar pada kesalahan konfigurasi, dan 18% berkas konfigurasi *open-source* teridentifikasi mengandung kerentanan keamanan yang serius [5].

Situasi ini mendorong pemanfaatan LLM untuk membantu menyusun maupun menjelaskan konfigurasi Kubernetes. Akan tetapi, LLM dapat menghasilkan kode yang tampak benar secara sintaksis namun keliru secara semantis [6], sebuah manifestasi umum dari kecenderungan halusinasi pada model generatif [7]. Upaya mitigasi melalui *retrieval-augmented generation* (RAG) berbasis vektor turut mengalami keterbatasan: pendekatan ini hanya mengandalkan kemiripan *embedding* sehingga gagal menangkap relasi kausal dan hierarkis yang menjadi inti logika domain Kubernetes [8].

Literatur *graph retrieval-augmented generation* (GraphRAG) mengatasi sebagian keterbatasan ini dengan menyipkan representasi pengetahuan terstruktur ke dalam proses *retrieval* [9], [10]. Meski demikian, dua celah teknis bertahan pada literatur yang ada. Pertama, mayoritas *pipeline* konstruksi *knowledge graph* (KG) memanfaatkan LLM sebagai ekstraktor, sehingga graf yang dihasilkan bervariasi antareksekusi dan sulit direproduksi [11]–[13]; keterbatasan serupa turut ditemukan pada pendekatan konstruksi KG

spesifik-domain lainnya [14]. Kedua, mekanisme *traversal* pada sistem GraphRAG yang ada umumnya menggunakan kedalaman tetap yang seragam untuk seluruh tipe kueri [15]–[17], padahal kebutuhan konteks berbeda signifikan antartipe pertanyaan pengguna [18], [19]. Kerangka evaluasi RAGAS [20] yang mengukur relevansi jawaban dan *faithfulness* turut menjadi rujukan metodologis evaluasi pada penelitian ini.

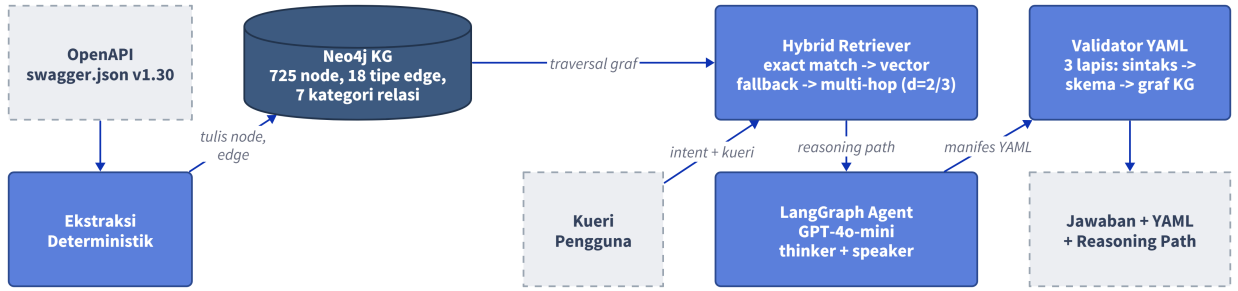
Berdasarkan kedua celah tersebut, penelitian ini mengajukan tiga kontribusi teknis. Pertama, *pipeline* konstruksi *knowledge graph* yang deterministik berbasis skema OpenAPI, sehingga hasil ekstraksi konsisten antareksekusi. Kedua, mekanisme *traversal* dengan kedalaman yang disesuaikan per tipe *intent* kueri (*intent-adaptive depth traversal*), alih-alih kedalaman tetap yang seragam. Ketiga, validasi struktural YAML tiga lapis berbasis *knowledge graph* yang berfungsi sebagai alternatif statis terhadap mekanisme *dry-run* pada kluster Kubernetes aktif. Bagian selanjutnya memaparkan perancangan sistem (§II), rancangan evaluasi (§III), hasil dan pembahasan (§IV), serta kesimpulan (§V).

II. PERANCANGAN SISTEM

Gambar 1 merangkum arsitektur sistem GraphRAG yang dibangun, mencakup zona konstruksi *knowledge graph* (luring) dan zona *retrieval*-generasi (saat kueri).

A. KONSTRUKSI KNOWLEDGE GRAPH DETERMINISTIK

Graf pengetahuan dibangun langsung dari blok *definitions* pada spesifikasi OpenAPI Kubernetes (*swagger.json*) melalui aturan transformasi yang eksplisit, menghasilkan 725 *node* dan 18 tipe *edge* dalam 7 kategori relasi semantik (struktural, *workload*, penyimpanan, konfigurasi, jaringan, RBAC, dan *autoscaling*). Konstruksi graf menggunakan dua



Gambar 1. Arsitektur sistem GraphRAG: konstruksi *knowledge graph* deterministik dari *swagger.json* (kiri), serta alur *retrieval* dan generasi jawaban dengan validasi YAML berbasis graf (kanan).

strategi komplementer: strategi deterministik yang mengubah setiap referensi *\$ref*, *allof*, *oneof*, dan *anyof* menjadi *edge* struktural secara langsung, serta strategi semi-deterministik yang membentuk relasi operasional yang relevan secara domain namun tidak tersurat eksplisit dalam skema, seperti relasi antara *Service* dan *Pod*. Karena seluruh aturan transformasi bersifat eksplisit dan dieksekusi tanpa komponen stokastik, graf yang dihasilkan konsisten antareksekusi dan setiap *edge*-nya dapat ditelusuri balik ke referensi skema asalnya, berbeda dengan pendekatan ekstraksi berbasis LLM. Representasi vektor semantik tiap *node* disimpan langsung sebagai properti graf di Neo4j, sehingga pencarian berbasis kemiripan dapat dieksekusi dalam graf yang sama tanpa *vector store* terpisah.

B. RETRIEVAL DENGAN KEDALAMAN ADAPTIF

Modul *retriever* mengimplementasikan *cascading retrieval* tiga tahap. Tahap pertama mencari *node* dengan nama yang persis sama dengan entitas hasil ekstraksi *intent* sebagai *seed node*, memberikan presisi tertinggi karena mengandalkan pencocokan eksak. Tahap kedua, yang hanya dieksekusi apabila tahap pertama gagal, mencari *node* dengan kemiripan vektor tertinggi terhadap kueri sebagai *fallback*. Tahap ketiga menelusuri graf secara rekursif dari *seed node* dengan batas kedalaman yang disesuaikan per tipe *intent*: kedalaman 2 untuk *intent explain* dan *followup* yang bersifat spesifik-*resource*, serta kedalaman 3 untuk *intent generate_yaml*, *trace_relationship*, dan *planning* yang membutuhkan konteks relasional lebih luas. Batas ini dipilih karena *node* pada kedalaman lebih dari 3 didominasi tipe generik yang dibagikan puluhan hingga ratusan *resource*, sehingga menurunkan presisi konteks tanpa menambah informasi yang relevan. Jalur relasional yang dilalui selama traversal disusun menjadi *reasoning path* dan diinjeksikan ke *prompt* sebagai bukti penalaran yang dapat diverifikasi pengguna.

C. VALIDASI YAML TIGA LAPIS BERBASIS KNOWLEDGE GRAPH

Ketika *intent* kueri diklasifikasikan sebagai *generate_yaml*, manifest YAML yang dihasilkan melalui tiga lapis validasi berurutan: lapis pertama memeriksa kebenaran sintaksis menggunakan *parser* YAML standar; lapis kedua memeriksa kesesuaian struktur terhadap skema resmi Kubernetes; dan

lapis ketiga melakukan kueri langsung ke *knowledge graph* untuk memverifikasi kelengkapan *field* wajib pada setiap *resource*. Lapis ketiga inilah yang menjadikan validasi ini bersifat *KG-grounded*: verifikasi kelengkapan *field* dilakukan tanpa memerlukan *dry-run* pada kluster Kubernetes aktif, karena informasi skema yang sama sudah tersedia dalam graf yang dibangun pada tahap konstruksi.

Seluruh mekanisme di atas dijalankan pada *pipeline* LangGraph dengan GPT-4o-mini berperan ganda sebagai *thinker* (klasifikasi *intent* dan ekstraksi entitas) maupun *speaker* (generasi jawaban), Neo4j sebagai penyimpanan graf sekaligus indeks vektor terpadu, dan basis data ringan sebagai memori percakapan berbasis sesi.

III. RANCANGAN EVALUASI

Evaluasi menggunakan 102 skenario uji (*fixture*) yang tersebar pada delapan kategori pertanyaan (*conceptual*, *yaml_gen*, *relationship*, *followup*, *realworld*, *planning*, *troubleshooting*, dan *command*), disusun berdasarkan hasil wawancara dengan tiga praktisi DevOps/SRE yang menggunakan Kubernetes dan LLM secara aktif. Ketiga praktisi juga memberi skor realisme terhadap sampel *fixture*, mengonfirmasi bahwa skenario berbasis situasi operasional konkret dinilai lebih representatif dibanding pertanyaan definitif murni.

Kinerja sistem diukur pada tiga dimensi. *Answer Quality* (AnsQ) merupakan rata-rata kemiripan semantik jawaban terhadap *ground truth*, validitas sintaksis YAML, dan kesesuaian skema. *Retrieval Quality* (RetQ) dioperasionalkan sebagai skor F1 antara himpunan *node* pada *reasoning path* sistem dan himpunan *node* relevan *ground truth*, mengikuti definisi *Precision/Recall* standar [21]:

$$\text{RetQ}_{F1} = \frac{2 \cdot P \cdot R}{P + R} \quad (1)$$

dengan *P* dan *R* masing-masing menyatakan Precision dan Recall *node reasoning path* terhadap *node* relevan *ground truth*. *Reasoning Quality* (ReaQ) memiliki dua komponen: *hop accuracy*, yaitu proporsi *edge ground truth* yang berhasil ditelusuri sistem,

$$\text{HopAcc} = \frac{|E_{\text{ditelusuri}} \cap E_{\text{ground truth}}|}{|E_{\text{ground truth}}|}, \quad (2)$$

serta *faithfulness*, yaitu proporsi klaim jawaban yang dapat dilacak ke konteks yang di-*retrieve* [20].

Ketiga dimensi ini diukur untuk membandingkan tiga sistem menggunakan *dataset*, model *speaker* (GPT-4o-mini, *temperature*=0,1), dan metrik yang identik: *Vanilla LLM* (tanpa konteks), *Vector RAG* (*cosine similarity top-5* tanpa *traversal* graf), dan GraphRAG (sistem lengkap). Signifikansi statistik diuji menggunakan uji Wilcoxon *signed-rank* dengan *paired bootstrap* 1.000 iterasi dan koreksi Holm-Bonferroni untuk perbandingan berganda.

IV. HASIL DAN PEMBAHASAN

A. PERBANDINGAN TIGA SISTEM

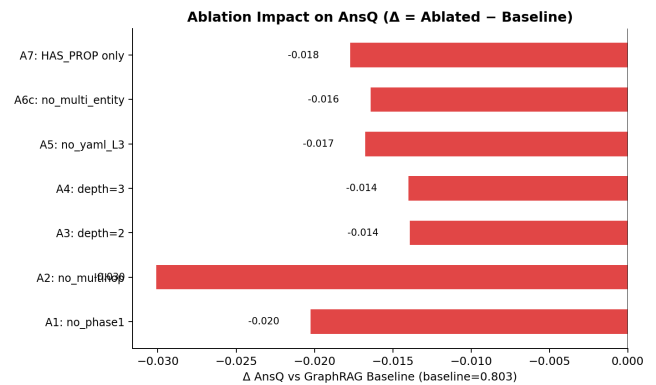
Tabel 1 merangkum skor ketiga sistem pada seluruh dimensi evaluasi beserta signifikansi statistiknya. RetQ merupakan dimensi paling diskriminatif: GraphRAG mencapai F1 0,71 dibanding 0,24 pada Vector RAG dan 0,00 pada Vanilla LLM, keduanya signifikan ($p < 0,001$). Keunggulan ini bersumber dari *traversal multi-hop* yang menjangkau *node* struktural antara (misalnya rantai *Deployment* → *DeploymentSpec* → *PodTemplateSpec*) yang tidak terjangkau oleh *Vector RAG*. Sebaliknya, AnsQ ketiga sistem relatif setara dan selisih GraphRAG terhadap Vector RAG tidak signifikan secara statistik ($p_{\text{Holm}} = 0,067$), mengindikasikan bahwa kualitas teks jawaban ditentukan oleh kapabilitas generatif LLM yang identik pada ketiga sistem, bukan oleh mekanisme *retrieval*.

B. KONTRIBUSI KOMPONEN SISTEM

Tabel 2 menyajikan hasil *ablation study* tujuh konfigurasi, sedangkan Gambar 2 memvisualisasikan penurunan RetQ dan *hop accuracy* yang paling menonjol. Menonaktifkan *multi-hop traversal* (A2) menghasilkan penurunan RetQ terbesar, mengonfirmasi bahwa komponen ini paling krusial: tanpa *traversal*, sistem hanya memperoleh *seed node* tanpa konteks dependensi skema. Menonaktifkan *exact match* (A1) juga menurunkan RetQ secara signifikan, menegaskan peran tahap ini sebagai gerbang ketepatan *seed node*. Mengganti 18 tipe *edge* semantik dengan satu tipe generik (A7) menurunkan RetQ maupun *hop accuracy* secara signifikan, membuktikan kontribusi taksonomi relasi terhadap kualitas *retrieval* dan penalaran. Sebaliknya, kedalaman tetap $d = 3$ (A4) tidak menurunkan RetQ secara signifikan namun meningkatkan *hop accuracy*, menunjukkan bahwa perilaku adaptif tidak sepenuhnya dapat digantikan satu nilai kedalaman tetap: kedalaman lebih dalam meningkatkan cakupan *edge* yang ditelusuri namun menurunkan presisi RetQ karena menyertakan *node* tambahan yang kurang relevan.

C. PEMBAHASAN: FAITHFULNESS DAN BATAS KEUNGGULAN

Faithfulness GraphRAG (0,31) lebih tinggi secara signifikan dibanding Vector RAG (0,17), namun nilai absolutnya berada jauh di bawah ambang ideal. Dekomposisi klaim jawaban mengungkap bahwa sekitar 56% klaim bersumber dari pengetahuan parametrik LLM, bukan dari konteks graf yang di-*retrieve*. Hal ini bukan cacat sistem, melainkan konsekuensi arsitektural: *knowledge graph* yang dibangun dari blok



Gambar 2. Penurunan RetQ F1 dan *hop accuracy* pada tiap konfigurasi ablati relatif terhadap *baseline*.

definitions hanya memodelkan relasi antar-*resource* bertipe objek sebagai *edge*, sedangkan nilai atribut skalar (seperti nama kontainer atau *image*) tidak dapat direpresentasikan sebagai simpul graf. Sistem karenanya dirancang *open-book*: LLM diizinkan melengkapi jawaban dengan pengetahuan parametrik ketika informasi tidak tersedia dalam graf. Analisis korelasi menunjukkan *faithfulness* berkorelasi lemah dan tidak signifikan terhadap AnsQ, menegaskan bahwa keduanya mengukur aspek yang berbeda: AnsQ mengukur kebenaran dan relevansi jawaban, sedangkan *faithfulness* mengukur keterlacakan klaim ke konteks yang di-*retrieve*.

Tiga keterbatasan turut membingkai cakupan temuan ini. Pertama, *knowledge graph* yang dibangun bersifat statis dan hanya merepresentasikan relasi struktural-skema dari blok *definitions*; relasi operasional seperti ikatan RBAC hanya terjangkau pada satu arah, dan status *runtime* klaster tidak terepresentasikan. Kedua, *retrieval* dibatasi pada maksimum tiga lompatan sehingga kueri yang membutuhkan konteks di luar jangkauan ini mengandalkan pengetahuan parametrik LLM secara *silent*, dengan kategori *relationship* memperoleh manfaat GraphRAG paling kecil di antara seluruh kategori kendati tetap positif. Ketiga, keunggulan GraphRAG tidak menjangkau seluruh dimensi evaluasi: AnsQ setara dengan Vector RAG, dan *faithfulness* mencerminkan keterbatasan cakupan nilai atribut dalam graf, bukan kegagalan mekanisme *retrieval* itu sendiri.

V. KESIMPULAN

Penelitian ini membangun sistem GraphRAG dengan tiga kontribusi teknis: konstruksi *knowledge graph* deterministik dari skema OpenAPI Kubernetes, mekanisme *retrieval* dengan kedalaman *traversal* adaptif per tipe *intent*, serta validasi struktural YAML tiga lapis berbasis graf sebagai alternatif terhadap *dry-run* klaster aktif. Evaluasi terhadap 102 skenario uji menunjukkan keunggulan GraphRAG yang signifikan pada dimensi *retrieval* (F1 0,71 dibanding 0,24, $\Delta +0,465$ terhadap Vector RAG) dan penalaran struktural, sementara kualitas teks jawaban tetap setara antarsistem karena ditentukan oleh kapabilitas generatif LLM yang identik. Temuan ini menegaskan bahwa kontribusi representasi graf

Tabel 1. Perbandingan Skor Tiga Sistem per Dimensi Evaluasi ($n = 102$)

Metrik	Vanilla LLM	Vector RAG	GraphRAG	Δ vs. Vector RAG	Δ vs. Vanilla LLM
AnsQ	0,7469	0,7984	0,8031	+0,005 (n.s.)	+0,056 (***)
RetQ (F1)	—	0,2437	0,7089	+0,465 (***)	+0,709 (***)
Hop Accuracy	N/A	N/A	0,7562	—	—
Faithfulness	N/A	0,1675	0,3055	+0,127 (***)	N/A
Syntactic Validity	0,8333	0,9333	1,0000	+0,036 (n.s.)	+0,143 (*)

— = tidak berlaku; N/A = tidak dapat dihitung (tanpa *retrieval* atau *traversal* graf). *** $p < 0,001$; * $p < 0,05$; n.s. = tidak signifikan (Wilcoxon + *paired bootstrap* 1.000 iterasi, koreksi Holm-Bonferroni).

Tabel 2. Ablation Study: Selisih terhadap Baseline GraphRAG ($n = 102$)

Konfigurasi	Δ AnsQ	Δ RetQ	Δ HopAcc
A1 <i>no_exact_match</i>	−0,020	−0,309***	−0,340***
A2 <i>no_multihop</i>	−0,030	−0,656***	−0,700***
A3 <i>depth=2 fixed</i>	−0,014	−0,123***	−0,147***
A4 <i>depth=3 fixed</i>	−0,014	−0,050	+0,145
A5 <i>no_yaml_layer3</i>	−0,017	−0,057**	+0,004
A6 <i>no_multi_entity</i>	−0,016	−0,011	−0,007
A7 <i>1 tipe edge</i>	−0,018	−0,152***	−0,232***

Δ = Ablasi − *baseline*; nilai negatif menunjukkan komponen berkontribusi positif. *** $p < 0,001$; ** $p < 0,01$ (Wilcoxon + *bootstrap*, $n = 102$); selebihnya tidak signifikan.

pada domain konfigurasi Kubernetes terletak pada ketertelusuran dan kelengkapan *retrieval* struktural, bukan pada kefasihan bahasa jawaban.

ACKNOWLEDGMENT

Naskah ini disusun dengan bantuan LLM untuk memadatkan teks dari dokumen Tugas Akhir penulis pertama; seluruh klaim teknis, data, dan hasil eksperimen berasal dari penelitian yang telah dilaksanakan dan divalidasi oleh penulis.

Pustaka

- [1] The Kubernetes Authors, "Kubernetes concepts," [Online]. Available: <https://kubernetes.io/docs/concepts/>, 2024, accessed: May 15, 2024.
- [2] Cloud Native Computing Foundation, "CNCF annual survey 2023," [Online]. Available: <https://www.cncf.io/reports/cncf-annual-survey-2023/>, 2023, accessed: Nov. 22, 2025.
- [3] Gartner, "Gartner says cloud will be the centerpiece of new digital experiences," [Online]. Available: <https://www.gartner.com/en/newsroom/press-releases/2021-11-10-gartner-says-cloud-will-be-the-centerpiece-of-new-digital-experiences>, 2021, accessed: Nov. 22, 2025.
- [4] J. Soldani, D. A. Tamburri, and W.-J. Van Den Heuvel, "The pains and gains of microservices: A systematic grey literature review," *Journal of Systems and Software*, vol. 146, pp. 215–232, 2018.
- [5] A. Rahman, A. Partho, P. Morrison, and L. Williams, "Security misconfigurations in open source Kubernetes manifests: An empirical study," *ACM Transactions on Software Engineering and Methodology*, vol. 32, no. 5, 2023.
- [6] Z. Liu, A. Nguyen, J. Chen, X. Zhang, Y. Li, and Y. Xiong, "Deep analysis of LLM-based code generation: Correctness, complexity, and security," *IEEE Transactions on Software Engineering*, pp. 1–20, 2024.
- [7] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.
- [8] Y. Wan, Z. Chen, Y. Liu, C. Chen, and M. Packianather, "Empowering LLMs by hybrid retrieval-augmented generation for domain-centric Q&A in smart manufacturing," *Advanced Engineering Informatics*, vol. 65, p. 103212, 2025.
- [9] H. Han, Y. Wang, H. Shomer, K. Guo, J. Ding, Y. Lei, M. Halappanavar, R. A. Rossi, S. Mukherjee, and X. Tang, "Retrieval-augmented generation with graphs (GraphRAG)," *arXiv preprint arXiv:2501.00309*, 2024.

- [10] X. He, Y. Tian, Y. Sun, N. V. Chawla, T. Laurent, Y. LeCun, X. Bresson, and B. Hooi, "G-Retriever: Retrieval-augmented generation for textual graph understanding and question answering," in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- [11] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, and X. Wu, "Unifying large language models and knowledge graphs: A roadmap," *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 7, pp. 3580–3599, 2024.
- [12] M. Hofer, D. Obraczka, A. Saeedi, H. Köpcke, and E. Rahm, "Construction of knowledge graphs: Current state and challenges," *Information*, vol. 15, no. 8, p. 509, 2024.
- [13] C. Ma, Y. Chen, T. Wu, A. Khan, and H. Wang, "Large language models meet knowledge graphs for question answering: Synthesis and opportunities," *arXiv preprint arXiv:2505.20099*, 2025.
- [14] B. Abu-Salih, "Domain-specific knowledge graphs: A survey," *Journal of Network and Computer Applications*, vol. 185, p. 103076, 2021.
- [15] H.-H. Yu, W.-T. Lin, C.-W. Kuan, C.-C. Yang, and K.-M. Liao, "GraphRAG-enhanced dialogue engine for domain-specific question answering: A case study on the civil IoT taiwan platform," *Future Internet*, vol. 17, no. 9, p. 414, 2025.
- [16] Z. Lu, H. Xu, A. Chen, S. Tang, J. Zhang, Y. Feng, W. Pan, and J. Huang, "HSG-RAG: Hierarchical knowledge base construction for embedded system development," *ACM Transactions on Design Automation of Electronic Systems*, vol. 30, no. 6, pp. 1–21, 2025.
- [17] R. Cao, H. Zhang, T. Huang, Z. Kang, Y. Zhang, L. Sun, H. Li, Y. Miao, S. Fan, L. Chen, and K. Yu, "NeuSym-RAG: Hybrid neural symbolic retrieval with multiview structuring for PDF question answering," *arXiv preprint arXiv:2505.19754*, 2025.
- [18] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-augmented generation for large language models: A survey," *arXiv preprint arXiv:2312.10997*, 2024.
- [19] A. Moreno-Cediel, E. Garcia-Lopez, A. Garcia-Cabot, and D. De-Fitero-Dominguez, "Optimising retrieval performance in RAG systems: A new growing window semantic chunking strategy to address weak semantic boundaries," *Knowledge-Based Systems*, vol. 331, p. 114896, 2025.
- [20] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated evaluation of retrieval augmented generation," *arXiv preprint arXiv:2309.15217*, 2023.
- [21] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, U.K.: Cambridge University Press, 2008.

JIHAN AURELIA adalah mahasiswa Program Studi Sistem dan Teknologi Informasi, Institut Teknologi Bandung, dengan fokus penelitian pada representasi pengetahuan berbasis graf untuk sistem *retrieval-augmented generation*.

DIMITRI MAHAYANA (Dr., Ir., M.Eng.) adalah dosen pembimbing pada Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung.

...